

Guía para el video de la entrevista

Guion para grabar la demo (≈ 21–26 minutos) de la sincronización bidireccional de facturas entre la API local (PostgreSQL) y QuickBooks Online, y cómo probar cada requisito del enunciado.

Idea central que hay que transmitir: PostgreSQL es la cola (transactional outbox). Cada cambio se guarda junto con su job en la misma transacción, un worker lo sincroniza, y QuickBooks avisa sus cambios por webhooks que también se guardan como jobs. Nada se pierde, nada se duplica y nada se sobrescribe en silencio.

1. Preparación antes de grabar

Checklist

- ☐ `nvm use` (Node 22) y `npm install`
- ☐ Postgres arriba y migrado: `npm run db:up` y `npm run db:migrate`
- ☐ Tests en verde (para mostrarlos al final): `npm test` → **161 pasan**
- ☐ QuickBooks conectado: `GET http://localhost:3000/quickbooks/status` → `connected: true` (si no: abrir `http://localhost:3000/quickbooks/connect` en el navegador)
- ☐ Webhooks reales: `cloudflared tunnel --url http://localhost:3000`, la URL `https://<...>.trycloudflare.com/quickbooks/webhooks` configurada en el portal de Intuit y el *Verifier Token* en `.env` (`QB0_WEBHOOK_VERIFIER_TOKEN`) y en `http-client.private.env.json`
- ☐ Una pestaña del navegador con el **sandbox de QuickBooks** (Sales → Invoices, filtro de fecha "All dates")
- ☐ GoLand con `client.http` abierto y una consola de base de datos con las consultas del apéndice A

Disposición de pantalla sugerida

Ventana	Para qué
Terminal 1	<code>npm run dev</code> (API, muestra los logs JSON de webhooks)
Terminal 2	<code>npm run dev:worker</code> (worker, muestra <code>sync.job.completed</code> , reintentos, etc.)
GoLand	<code>client.http</code> (requests con ▶ y pestaña <i>Tests</i>) + consola SQL
Navegador	sandbox de QuickBooks + <code>docs/system-design.png</code> (o el <code>.excalidraw</code> abierto en <code>excalidraw.com</code>)

Consejo: los logs son JSON de una línea. Con la terminal ancha se leen bien los campos `message`, `entity_id`, `result`, `error_class`.

2. Guion con tiempos

#	Sección	Tiempo
0	Introducción y decisión clave	1 min

#	Sección	Tiempo
1	Arquitectura (imagen)	3 min
2	Creación de facturas	2 min
3	Actualizaciones	2 min
4	Eliminación / anulación (void)	2 min
5	Estado de pago y pagos parciales	3 min
6	Eventos duplicados	2 min
7	Ediciones en conflicto en ambos sistemas	3 min
8	Fallos, timeouts y borrados a mano (resiliencia)	3 min
9	Tests	2 min
10	Trade-offs y qué haría en producción	1–2 min

0. Introducción (1 min)

Qué decir:

"El problema es mantener facturas sincronizadas en ambos sentidos entre mi API y QuickBooks, con eventos que pueden llegar duplicados, tarde o desordenados, APIs que fallan y usuarios editando en los dos lados. Mi decisión principal fue usar PostgreSQL como cola en lugar de un broker: el cambio y su job se guardan en la misma transacción, así que es imposible guardar algo sin encolarlo."

1. Arquitectura (3 min)

Mostrar el diagrama estilo pizarra `docs/system-design.png` (o `docs/system-design.excalidraw` abierto en `excalidraw.com`, para señalar en vivo) y recorrer las flechas numeradas. `docs/architecture.png` tiene el mismo flujo con más detalle (rutas, tablas, estados de los jobs), por si preguntan:

1. El cliente crea o cambia una factura o pago (REST).
2. La API guarda **el cambio + el job + NOTIFY** en una transacción y responde sin esperar a QuickBooks.
3. El worker se despierta en milisegundos (LISTEN/NOTIFY; polling cada 5 s como respaldo) y toma el job con `FOR UPDATE SKIP LOCKED`, claim token y lease.
4. El worker empuja a QuickBooks (create con `requestid`, update con `SyncToken`, void/delete, pagos). Nunca hay una transacción abierta durante una llamada HTTP.
5. Un cambio en QuickBooks llega como **webhook firmado**: la API verifica la firma, guarda el evento y su job, y responde 200 **solo después del commit**.
6. El worker **vuelve a pedir la entidad completa** a QuickBooks y la aplica. La reconciliación (CDC) atrapa cualquier webhook perdido.

Si preguntan por el polling (dónde está y para qué, si ya hay NOTIFY), abrir `src/worker.ts`:

- **El bucle principal** (`main`, `src/worker.ts:153`): en cada vuelta recupera leases vencidos, reconcilia con QuickBooks si toca (cada 5 min), corre el chequeo de consistencia si toca (cada 24 h), **vacía la cola** con `claimNextJob` y después espera con `waitForWork(POLL_MS)`.

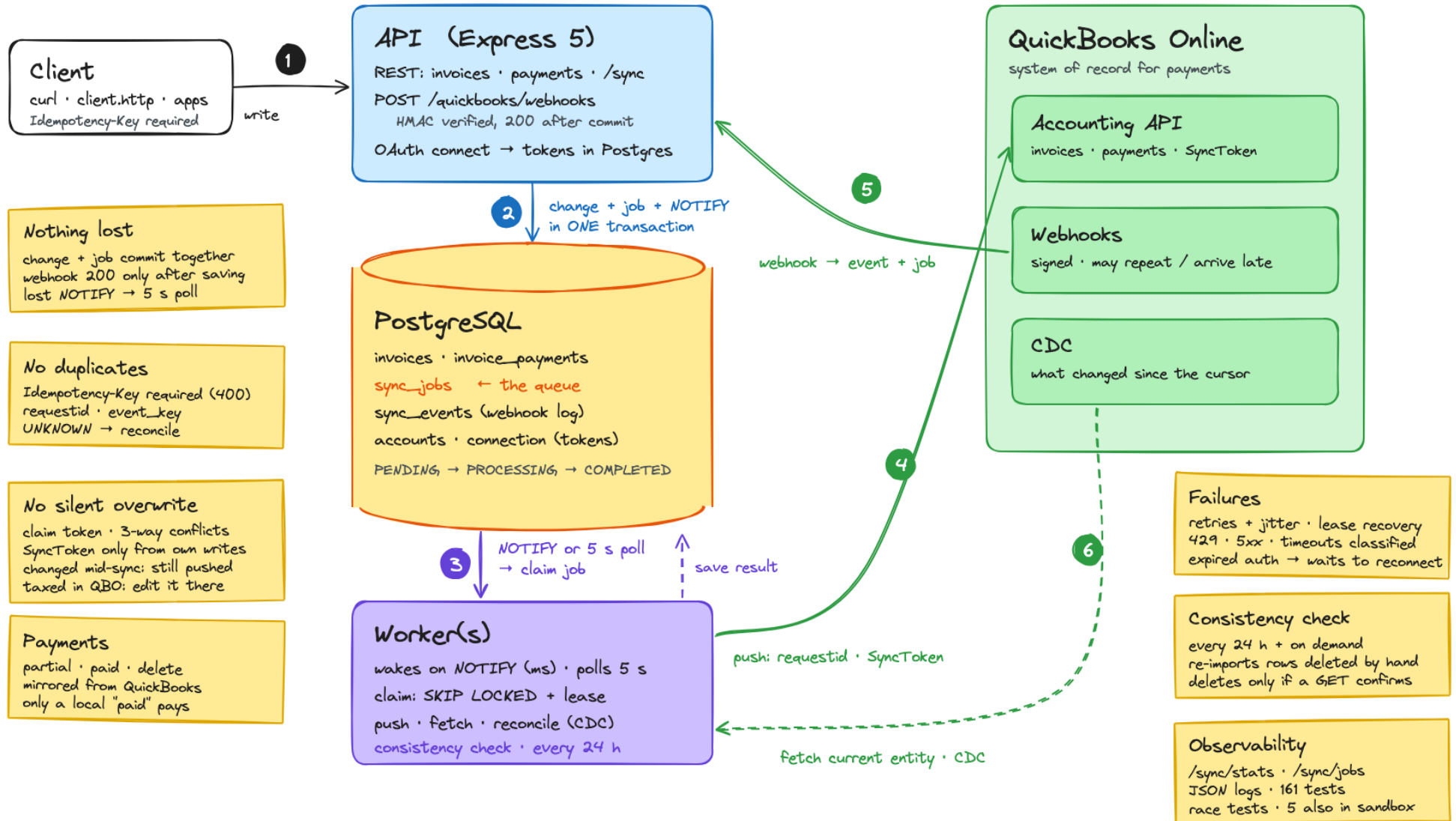
- **La espera** (`waitForWork`, `src/worker.ts:51`): un `setTimeout` de `SYNC_POLL_INTERVAL_SECONDS` (5s) que un `NOTIFY` corta antes. Sin `NOTIFY` el timer vence y el bucle vuelve a mirar la cola: **eso es el polling**.
- **La consulta del polling** (`claimNextJob`, `src/sync.repository.ts:151`): un solo `UPDATE ... WHERE id = (SELECT ... FOR UPDATE SKIP LOCKED)` que toma el próximo job `PENDING` vencido, saltando facturas que ya tienen un job anterior o en curso, y lo marca `PROCESSING` con claim token y lease.
- **Polling a QuickBooks** (`reconcile`, `src/worker.ts:106` → `enqueueRemoteChanges` en `src/sync.reconcile.ts`): cada `SYNC_RECONCILE_INTERVAL_SECONDS` le pregunta al CDC qué cambió desde el cursor y lo encola; es el respaldo de los webhooks.

Qué decir: *"NOTIFY da la latencia, el polling da la garantía. NOTIFY no es durable: si el worker está caído o se corta su conexión, el aviso se pierde. El job no, porque está en la tabla; la próxima vuelta del polling lo encuentra. Y lo mismo hacia el otro lado: los webhooks dan la latencia, el CDC la garantía."*

Remarcar las garantías (notas amarillas; panel **Guarantees** en el detallado): nada perdido, nada duplicado, nada sobrescrito en silencio.

Invoices $\rightleftharpoons$ QuickBooks Online

two-way sync · PostgreSQL is the queue (transactional outbox) · no message broker



Invoices API ↔ QuickBooks Online

Two-way sync with PostgreSQL as the queue (transactional outbox) · Node.js · TypeScript · Express 5 · Slonik · zod · no message broker

Clients

curl · client.http (GoLand)
apps using the REST API
Idempotency-Key required on POST

How a change travels

- 1 A client creates or changes an invoice or payment (REST).
- 2 The API saves the change and its sync job in ONE transaction, sends NOTIFY (delivered on commit) and answers without waiting.
- 3 On NOTIFY (ms) or its 5 s poll, a worker claims the job (SKIP LOCKED, claim token, lease). Jobs of one invoice run one at a time, in order.
- 4 It pushes to QuickBooks: create with requestid + reference, sparse update with SyncToken, void/delete, payments. Result + job completion: one short transaction (claim token checked).
- 5 A change in QuickBooks arrives as a signed webhook: the API verifies it, stores sync_event + job, and answers 200 only after the commit.
- 6 The worker fetches the current entity and applies it: older/duplicate versions skipped, conflicts flagged, balance · paid · sent · void updated. Reconciliation (CDC) catches any webhook that was missed.

Tests

161 tests on a separate invoices_test DB, with an in-memory fake QuickBooks and fault injection (lost responses, timeouts, stale versions)

+ race tests: a local change made while a job runs or waits

+ integration tests against the real QuickBooks sandbox, incl. 5 of the races

API — Express 5 (src/server.ts)

Validates with zod · never calls QuickBooks while handling invoices, payments or webhooks

POST · PATCH · DELETE /invoices (POST: Idempotency-Key)
POST · DELETE /invoices/:id/payments (POST: Idempotency-Key)
GET /accounts /sync/jobs /sync/stats /sync/events
POST /sync/jobs/:id/retry /sync/conflicts/:id/resolve
POST /sync/consistency-check (all ids vs QuickBooks)
GET /quickbooks/connect → /callback (OAuth 2.0)
POST /quickbooks/webhooks (HMAC-SHA256 verified)

One transaction: the change + its sync job + NOTIFY → respond without waiting

2 change + job + NOTIFY
(atomic, outbox)

PostgreSQL (Docker · migrations with node-pg-migrate)

Business data

invoices
NUMERIC money, version, synced snapshot, sync_status, conflict, quickbooks id + realm

invoice_payments
recorded here + mirrored from QuickBooks

quickbooks_accounts
local copy of the chart of accounts

Synchronization

sync_jobs — the queue
claim token, lease, attempts, next_run_at
entity_key: one job per invoice at a time

sync_events
webhook / reconciliation log, event_key unique

quickbooks_connection
tokens, CDC cursor

Job states: PENDING → PROCESSING → COMPLETED · FAILED (retry via API)
UNKNOWN = create sent, response lost → reconciled, never resent blindly

3 NOTIFY wakes · or 5 s poll
claim job
save result + complete job

Worker(s) (src/worker.ts · run 1..N)

Wake: LISTEN sync_jobs (ms) · poll every 5 s: a missed NOTIFY loses no job

Claim: FOR UPDATE SKIP LOCKED + claim token + lease · no transaction during HTTP

Outbound: invoices create · update · sent · paid · void · delete
payments create · delete (load the latest local state)

Inbound: fetch the current entity · SyncToken ordering · 3-way conflict check
balance / paid / sent / void · deletions · payments · accounts

Reconcile (every N min): CDC catch-up · full import if empty, > 30 days or capped
UNKNOWN creates searched by reference · expired leases recovered

Consistency (every 24 h + on demand): all ids vs QuickBooks · re-import missing rows (e.g. deleted by hand) · delete only if a GET confirms

push (requestid)

fetch current · CDC

QuickBooks Online (Intuit)

Sandbox company · system of record for payments

OAuth 2.0

tokens in Postgres, refresh with compare-and-set

Accounting API

Invoice · Payment · Customer · Item · Account
sparse updates with SyncToken · throttled 150 ms

Webhooks

Invoice · Payment · Account (legacy or CloudEvents)
dev: through a Cloudflare tunnel

Change Data Capture (CDC)

what changed since the cursor (≤ 30 days)
+ paginated queries for the full import

Guarantees

Nothing lost

- change + job commit together (outbox)
- webhook answered 200 only after saving
- dead worker: lease expires, job recovered
- missed NOTIFY: the next 5 s poll finds the job

Nothing duplicated

- Idempotency-Key required on POST (else 400)
- requestid + reference in PrivateNote
- lost response → UNKNOWN → reconciliation
- webhook redelivery: event_key unique
- only a "paid" set here records a payment

Nothing overwritten silently

- claim token: a stale worker can't finish a job
- SyncToken only from our own writes: no loops
- 3-way check → conflict, resolved via the API
- taxed in QuickBooks: content edited there only
- changed mid-sync: still pushed (paid, void, delete)

Failures handled

- retries with exponential backoff + jitter
- 429 · 5xx · timeouts · stale version classified
- FAILED / UNKNOWN jobs: inspect and retry
GET /sync/jobs · POST /sync/jobs/:id/retry
- authorization expired: jobs wait for reconnect

Observability

- /sync/stats: pending, failed, unknown, oldest pending, avg processing time
- JSON logs with job, event, entity, realm, attempt, duration, error class
- consistency check: repairs rows changed outside the app (e.g. deleted by hand)

2. Creación de facturas (2 min)

Local → QuickBooks

1. `client.http` → **1. Create an invoice** (►). Señalar el header `Idempotency-Key: {{$random.uuid}}`: es **obligatorio** (sin él, 400). Mostrar la respuesta: `sync_status: "pending"`, `amount: "2006.00"` (decimal como texto, nunca float).
2. En la terminal del worker aparece `sync.job.completed` casi al instante.
3. **2. A few seconds later** → `sync_status: "synced"` y `quickbooks_id`.
4. Mostrarla en el sandbox de QuickBooks.
5. Opcional, para impresionar: la consulta **A3** muestra que el worker tomó el job en **~5 ms** (NOTIFY).

QuickBooks → local

1. En QuickBooks: **+ New** → **Invoice**, guardar.
2. En la terminal de la API: `sync.webhook.event ... outcome: "enqueued"`; en el worker: `imported from QuickBooks`.
3. `GET /invoices` (o consulta A1): la factura aparece localmente.

Qué decir: "El webhook no trae la factura completa, solo 'cambió la 123'. Por eso el worker siempre vuelve a pedirla; así un aviso viejo o incompleto nunca pisa datos nuevos."

3. Actualizaciones (2 min)

Local → QuickBooks: `client.http` → **Update the invoice created in 1** (cambiar `amount` en el body, por ejemplo `"2100.00"`). Refrescar la factura en QuickBooks: nuevo total.

QuickBooks → local: en QuickBooks cambiar la **fecha de vencimiento** de esa factura y guardar. Ver en la consola `due_date` actualizado (A1) y el job INBOUND con `result: "applied QuickBooks changes"` (A2).

Qué decir: "Cada actualización a QuickBooks lleva su `SyncToken` (control de concurrencia optimista). Si alguien cambió la factura en medio, QuickBooks la rechaza y el job se reintenta con la versión nueva."

4. Eliminación y anulación (2 min)

Acción	Resultado
<code>DELETE /invoices/:id</code> (sin pagos)	Se borra en QuickBooks; cuando llega el aviso, la fila local se elimina
<code>DELETE /invoices/:id</code> (con pagos)	En QuickBooks se anula (void) en vez de borrar (conserva el historial de pagos)
<code>PATCH {"status": "void"}</code>	Se anula en QuickBooks
Void en QuickBooks (More → Void)	Local: <code>status = void</code> , <code>balance = 0.00</code> , se conserva el monto facturado
Delete en QuickBooks (More → Delete)	Local: la fila se elimina

Demo rápida: anular una factura en QuickBooks y mostrar A1 (`status = void`, `amount` intacto, `voided_at`).

Qué decir: "QuickBooks no tiene un campo 'anulada': pone los montos en cero y agrega 'Voided' a la nota. Lo verifiqué contra el sandbox real y lo detecto aunque el aviso no diga 'Void'."

5. Estado de pago y pagos parciales (3 min)

Correr la sección **P1–P13** de `client.http`, en orden (cada request tiene tests; ver pestaña *Tests*):

- **P2** pago parcial de 30.00 → **P5–P6**: sincronizado, balance **70.00** (viene de QuickBooks).
- **P3** reintento con el mismo `Idempotency-Key` → 200, **mismo pago** (no se paga dos veces). La clave también es **obligatoria** en los pagos: sin ella, 400.
- **P4** pagar 80.00 cuando quedan 70.00 → **422**. Lo que queda es el monto actual menos los pagos (sincronizados o no): sigue un cambio local de monto aunque no se haya sincronizado, y si la factura se marcó pagada localmente no queda nada (su job paga el saldo). Un pago de 0.00 → **400**.
- **P7** `status: paid` → el worker paga el resto → **P9**: `paid`, balance 0.00.
- **P10** volver a `sent` → **409** (hay que borrar un pago). Tampoco se puede editar una factura anulada (**409**): QuickBooks no lo permite.
- **P11–P12** borrar el pago de 30.00 → se borra en QuickBooks, balance vuelve a 30.00, `sent`.

Luego **en QuickBooks**: *Receive payment* sobre otra factura → aparece en `GET /invoices/:id/payments` y la factura pasa a `paid` si queda en 0.

Detalle para mencionar: solo un "paid" puesto **acá** genera un pago en QuickBooks. Si el pago se borra en QuickBooks, o se sube el monto de una factura pagada, la factura vuelve a `sent`: nunca se registra un pago que nadie hizo.

También sirve mostrar "**Mark as sent**" en QuickBooks → la factura local pasa de `draft` a `sent` (detalle interesante: ese cambio **no** modifica el `SyncToken` en QuickBooks, lo descubrí probando).

6. Eventos duplicados (2 min)

1. **D1–D7** en `client.http`: el mismo POST con la misma `Idempotency-Key` devuelve la **misma factura** (D2 → 200); sin clave el POST se **rechaza con 400** (D3), porque la clave es obligatoria; con una clave nueva sí se crea un duplicado real (D4, un cliente que genera otra clave al reintentar), y **D5** (`/quickbooks/duplicates`) lo detecta.
2. **W1–W2** (simular webhook firmado, entorno `dev`): enviar W2 **dos veces** con el mismo Id. El segundo job termina con `result: "already up to date"` (consulta A2): el `SyncToken` ya estaba aplicado, no cambia nada.
3. Mostrar `GET /sync/events`: cada aviso tiene un `event_key` **único**; una reentrega idéntica de Intuit se descarta (`outcome: "duplicate"` en el log).

Qué decir: "Tengo tres capas: `Idempotency-Key` para reintentos del cliente, `event_key` único para reentregas del webhook, y el `SyncToken` para que un aviso repetido, viejo o desordenado no aplique nada. El `Idempotency-Key` es obligatorio en todo POST que crea algo (facturas y pagos): si fuera opcional, el cliente que lo olvida es justo el que duplica una factura al reintentar tras un timeout. Mejor un 400 claro que un duplicado silencioso. Lo único que no puedo evitar es un cliente que genera otra clave al reintentar (D4); para eso está el chequeo de duplicados."

7. Ediciones en conflicto en ambos sistemas (3 min)

Cómo reproducirlo en vivo (determinístico):

1. **Detener el worker** (Ctrl+C en la terminal 2).
2. Localmente: `PATCH /invoices/:id` con `{"amount": "150.00"}` → queda `pending`.
3. En QuickBooks: cambiar el **monto** de esa misma factura a otro valor (por ejemplo 175) y guardar. El webhook llega y se guarda (se ve en la terminal de la API), pero nadie lo procesa aún.
4. **Arrancar el worker** (`npm run dev:worker`).
5. Mostrar la factura (A1 o `GET /invoices/:id`): `sync_status: "conflict"` y `sync_conflict` con las tres versiones: `local` (150.00), `remote` (175.00), `base` (el último acuerdo), y `fields: ["amount"]`. Ninguno de los dos lados fue sobrescrito.
6. Resolver: `client.http` → **Resolve a conflict** con `{"keep": "remote"}` (o `"local"`, que empuja la versión local contra la versión actual de QuickBooks). Con `"remote"` se toma el contenido de QuickBooks, pero lo demás que cambió localmente durante el conflicto (un borrado, un void, "paid") **igual se empuja**: se encola un job que lo compara contra QuickBooks.

Qué decir: "Comparo tres versiones: el snapshot del último acuerdo, la local y la de QuickBooks. Si solo cambió un lado, se aplica; si los dos hicieron el mismo cambio, converge; si cambiaron distinto, es un conflicto y no toco datos financieros. El saldo y el estado de pago siempre los manda QuickBooks, porque ahí se registran los pagos. Y no todo cambio en ambos lados es conflicto: si localmente solo cambié el estado (la marqué pagada) y en QuickBooks cambiaron la fecha, tomo la fecha de QuickBooks y empujo el pago encima."

8. Fallos, timeouts y borrados a mano (3 min)

En vivo: nada se pierde con el worker caído

1. Detener el worker. Crear dos facturas con `client.http`.
2. `GET /sync/stats` → `pending: 2`, `oldest_pending_seconds` creciendo. QuickBooks no tiene nada.
3. Arrancar el worker → las procesa enseguida (logs + stats en 0).

Qué decir: "Los NOTIFY de esas dos facturas se perdieron, porque nadie estaba escuchando. Las tomó la primera vuelta del bucle de polling del worker (`src/worker.ts:153`), que al arrancar mira la cola antes de esperar. Por eso el polling no es opcional: NOTIFY solo acelera."

En vivo: alguien borra una factura directamente en la base

1. Elegir una factura sincronizada que tenga un pago (consulta A5) y borrarla en la consola SQL: `DELETE FROM invoices WHERE id = 123;`. Sus pagos locales se borran en cascada y QuickBooks no se entera (el borrado se saltó la API y su outbox).
2. `client.http` → **Consistency check, report only** (►). En el log de la respuesta: `missing locally: ["<id QuickBooks>"]`. Remarcar que este modo **no cambia nada**, solo reporta.
3. `client.http` → **Consistency check, repairing** (►). En el log: `differences: 1 | repair jobs queued: 1`, y la pestaña *Tests* en verde.
4. Unos segundos después (consultas A1 y A5): la factura **volvió** con un id local nuevo, el mismo `quickbooks_id`, su balance y **sus pagos**.
5. Opcional: volver a correr **Consistency check, report only** → `missing locally: []`, todo consistente.

Qué decir: "El CDC solo ve lo que cambió. Para lo que nadie avisa, como un borrado a mano o un backup restaurado, hay un chequeo de consistencia completo cada 24 horas que compara todo y repara por la misma cola. Antes de borrar algo local porque no está en QuickBooks, lo confirmo con un GET directo."

Con tests: lo que no se puede reproducir a mano (fault injection):

```
npx tsx --env-file=.env.test --import ./test/setup.ts --test test/sync.outbound.test.ts
```

Mencionar los casos:

- **Timeout después de que QuickBooks creó la factura** → el job queda UNKNOWN, **no se reenvía**; la reconciliación busca la factura por su referencia (local-invoice-<id>-<fecha> en el PrivateNote) y la vincula.
- Si no estaba → se reenvía con el mismo requestid (QuickBooks deduplica igual).
- 429, 5xx, timeouts → reintentos con backoff exponencial + jitter; errores de validación → FAILED directo.
- Worker muerto a mitad de un job → el lease expira y otro worker lo retoma; el claim token impide que el worker viejo lo complete, y también que registre un error: si su request falla tarde, no marca failed una factura que el otro worker ya sincronizó.
- Refresh token vencido (100 días) o revocado → cuenta como "no conectado": los jobs esperan sin gastar intentos y se despiertan al reconectar.
- GET /sync/jobs?status=FAILED y POST /sync/jobs/:id/retry para operar a mano.

9. Tests (2 min)

```
npm test          # 161 tests contra la base invoices_test (nunca la principal)
npm run test:sandbox # contra el sandbox real de QuickBooks, con carreras y fixes de las revisiones (detener el worker antes)
```

Mostrar que hay un **QuickBooks falso en memoria** (test/helpers.ts) y **fault injection** (src/faults.ts) para simular respuestas perdidas, timeouts y versiones viejas.

Carreras encontradas en code review (test/sync.races.test.ts):

```
npx tsx --env-file=.env.test --import ./test/setup.ts --test test/sync.races.test.ts
```

Una revisión de código encontró 8 carreras entre un cambio local y un job que ya estaba corriendo o esperando (y al corregirlas apareció una más). Para cada una escribí primero un test que **la reproducía (fallaba)**, y después el fix. Para simular "el usuario cambia algo mientras el worker está hablando con QuickBooks", el QuickBooks falso tiene un hook, duringNext, que corre un cambio local en medio de un request. Ejemplos para mencionar:

- **Marcada pagada mientras el worker empujaba otro cambio** → antes se guardaba como sent y el pago nunca llegaba a QuickBooks. Ahora un "paid" local no empujado se mantiene (syncedStatus).
- **Borrada mientras el create estaba UNKNOWN** → al vincularla se marcaba como sincronizada y el borrado se perdía. Ahora solo se marca sincronizada si QuickBooks tiene todo: mismo contenido y nada pendiente de empujar (borrado, void, pago, enviada) (matchesQuickBooks).

- **Un pago sincronizado antes de un aviso pendiente de QuickBooks** → el pago adelantaba el SyncToken y la edición hecha en QuickBooks se descartaba como "ya aplicada". Ahora solo el job inbound mueve el SyncToken.
- **Falso conflicto** cuando localmente solo cambió el estado y en QuickBooks el contenido.

Qué decir: *"Los bugs de sincronización casi nunca están en el camino feliz, están en el orden de los eventos. Por eso cada carrera tiene su test que la reproduce: si alguien la reintroduce, el test falla."*

Las mismas carreras contra el sandbox real (`npm run test:sandbox` , con el worker detenido): cinco de ellas también corren contra QuickBooks de verdad, no solo contra el falso:

- marcada pagada mientras el worker empuja otro cambio → el pago llega a QuickBooks (saldo 0);
- un pago sincronizado antes de una edición pendiente de QuickBooks → la edición igual se aplica;
- borrada mientras el create estaba UNKNOWN , resuelta por la reconciliación → borrada en QuickBooks;
- anulada localmente mientras se aplica antes una edición de QuickBooks → anulada en QuickBooks;
- solo cambió el estado local y el contenido en QuickBooks → sin conflicto, el pago se empuja.

Para "cambiar algo en medio de un request" contra la API real, el test envuelve el cliente de QuickBooks (`withChangeDuring`) y hace el cambio local justo antes de que salga el request. Los tests borran sus pagos al final, así la limpieza borra las facturas en vez de anularlas y no queda nada en el sandbox.

Qué decir: *"El QuickBooks falso me deja reproducir cualquier orden de eventos; el sandbox confirma que el fix funciona con el comportamiento real de QuickBooks, que no siempre es el documentado."*

Segunda revisión de código: encontró 9 problemas más, y los traté igual: primero un test que lo reproducía, después el fix. Los más interesantes para contar:

- **Pagos que nadie hizo:** un "paid" viejo, que venía de QuickBooks, se volvía a pagar ante cualquier edición local. Ahora solo paga un "paid" puesto acá (el saldo local, leído de QuickBooks, todavía es mayor que 0).
- **Worker con lease vencido** que marcaba `failed` una factura ya sincronizada por otro worker.
- **Refresh token vencido o revocado** tratado como error interno: cada job gastaba sus 8 intentos y quedaba `FAILED` . Ahora espera a que se reconecte.
- **CDC tiene límites** (30 días, 1000 objetos por entidad): pasado eso se hace una importación completa.
- **Reintentos concurrentes del mismo pago** daban 500: ahora el segundo recibe el mismo pago. Este test al principio pasaba por casualidad (dos `Promise.all` no siempre se pisan); lo reescribí para que la carrera sea determinística, y ahí falló como se esperaba.

Qué decir: *"Un test que pasa no prueba nada si no lo vi fallar primero."*

Y contra el sandbox real, tres fixes de esta revisión:

- un pago borrado en QuickBooks no se vuelve a pagar por una edición local;
- subir el monto de una factura pagada no paga la diferencia (queda `sent` con saldo 10);
- **QuickBooks rechaza de verdad un cursor de CDC de más de 30 días:** era la suposición del fix, y hasta ahí solo la simulaba el QuickBooks falso. Después el test atrasa el cursor 40 días y comprueba que la reconciliación se pone al día igual (con una importación completa).

El refresh token vencido no se prueba contra el sandbox: habría que romper la conexión real.

Qué decir: *"El falso reproduce lo que yo creo que hace QuickBooks; el sandbox comprueba que QuickBooks realmente lo hace."*

Limpieza después de los fixes: con todos los tests en verde, una pasada de simplificación (reuso, complejidad, eficiencia y si cada fix estaba a la profundidad correcta). Lo más útil para contar:

- la regla "qué estado local todavía no tiene QuickBooks" (paid, sent, void) estaba **copiada en cuatro lugares** y las copias ya habían divergido; ahora es una sola función, `statusLeftToPush` ;
- el mismo "encolar el push de esta factura" estaba copiado cuatro veces: ahora es `enqueueInvoicePush` ;
- un fix que se había aplicado en un solo lugar (no adelantar el SyncToken con una lectura posterior al pago) se generalizó a `saveSynced` ;
- las validaciones del PATCH (factura anulada, "des-pagar") pasaron al `UPDATE` mismo, así no pueden correr carrera con un void que llega al mismo tiempo.

Qué decir: *"Primero que funcione y esté probado, después que quede simple. Los tests me dejaron refactorizar tranquilo: la limpieza no cambió ningún comportamiento, y cuando rompí algo (un `NULL` en un `WHERE`), un test de carreras lo agarró al instante."*

La limpieza generalizó un fix (el SyncToken solo sale de nuestra propia escritura, nunca de la lectura posterior a un pago), así que también tiene su test, en los dos lados:

- **con el QuickBooks falso** (carrera 13): alguien edita la factura en QuickBooks justo entre nuestro pago y la lectura que le sigue, y la edición igual se aplica localmente. Como el fix salió de la limpieza, ningún test lo había visto fallar: volví temporalmente al comportamiento viejo y el test falló (la fecha quedaba en la anterior); con el fix pasa;
- **contra el sandbox real** (`npm run test:sandbox`): el mismo escenario con `withChangeDuring` , editando la factura en QuickBooks de verdad en ese instante.

Tercera revisión: 5 hallazgos. Dos dependían de cómo se comporta QuickBooks y el revisor no lo había confirmado, así que **primero lo comprobé en el sandbox** con un script descartable (crea y borra una factura y un pago):

- *"un pago anulado deja líneas con monto 0 y rompe el CHECK de la base"* → **falso**: QuickBooks devuelve el pago anulado **sin líneas**, y el espejo simplemente borra sus filas locales. No había nada que arreglar;
- *"el SyncToken de una cuenta no cambia cuando cambia su saldo"* → **cierto**: el saldo de Cuentas por Cobrar cambió y el SyncToken siguió en "0". Las cuentas nunca actualizaban su saldo. Y el test mostró algo más: una versión **más vieja** sí pisaba la cuenta. Ahora se actualiza salvo que el SyncToken sea anterior.

Los otros tres, con su test que primero falló:

- borrada localmente mientras el create estaba `UNKNOWN` y anulada en QuickBooks → el borrado igual llega;
- el pago llegó a QuickBooks pero la lectura posterior falló (503) → el reintento ya no lo marca `failed` ;
- cambiar el monto de una factura que todavía no está en QuickBooks también cambia su saldo.

Qué decir: *"Una revisión puede equivocarse. Antes de arreglar algo que depende de QuickBooks, lo compruebo contra QuickBooks: un hallazgo era falso y otro era peor de lo que decía."*

Cuarta revisión (a fondo): 10 hallazgos, 9 arreglados con su test que primero falló. Para contar:

- **Impuestos:** la empresa del sandbox cobra sales tax, y QuickBooks devuelve `TotalAmt` con el impuesto incluido (empujo 100, QuickBooks dice 108). Mi modelo es "una factura = una línea con un monto", así que local y base nunca coincidían y cada edición en QuickBooks terminaba en conflicto. **Fue una decisión de diseño:** una factura con impuesto en QuickBooks (`taxed_in_quickbooks`) queda de solo lectura acá en su contenido (409) y su contenido lo manda QuickBooks; estado, pagos y borrado siguen sincronizando. Modelar impuestos de verdad requiere modelar líneas, y eso queda fuera de alcance.
- **Un token por request:** la API compartía un solo cliente OAuth entre requests concurrentes; uno podía salir con el token de otro. Ahora cada request tiene el suyo. El test lo hace determinístico con el throttle.
- **Un cambio que llega mientras corre el job anterior:** la API no encolaba (veía un job esperando) y el worker ya había leído la versión vieja. Ahora todo job que termina sin empujar vuelve a encolar si la versión cambió. Para el test hubo que mantener una transacción abierta mientras corre el worker (`AsyncResource`).
- Más chicos: un pago de una factura ya borrada ya no se envía; un pago con dos líneas para la misma factura se suma; si el CDC no alcanza, la importación completa ahora también aplica los borrados (con el chequeo de consistencia); una clave de idempotencia reusada en otra factura da 409.

Qué decir: "Cuando un hallazgo toca el modelo, no lo parcho: elijo explícitamente qué no soporto y lo hago seguro. Las facturas con impuesto se editan en QuickBooks."

Y contra el sandbox real, tres de esta revisión:

- una factura de ejemplo **con impuesto** del sandbox (solo se lee, no se toca): se reconoce por su `TxnTaxDetail` real, lo que confirma el campo que asume el QuickBooks falso; su copia local está marcada y un PATCH del monto da **409** sin cambiar nada;
- pago registrado y después la factura borrada → en QuickBooks la factura queda **borrada**, no anulada, y sin pago;
- un pago con **dos líneas para la misma factura** (20 + 10), hecho directo en QuickBooks → se lista como 30.00.

10. Trade-offs y producción (1–2 min)

- **Por qué Postgres y no RabbitMQ/Kafka:** atomicidad cambio+job sin dual write, un sistema menos. Un broker tendría sentido con fan-out a muchos consumidores, decenas de miles de mensajes por segundo o replay de eventos (y aun así con un outbox como fuente).
- **En producción:** tokens cifrados, multi-empresa (el esquema ya está por `realm_id`), estado OAuth fuera de memoria para varias instancias de la API, heartbeat de leases, alertas sobre `/sync/stats` , retención de jobs.

3. Cómo se cubre cada requisito

Requisito	En vivo	Tests (nombre)
Creación	<code>client.http</code> 1–3; crear en QuickBooks → aparece local	<code>creates the invoice in QuickBooks and stores the mapping</code> · <code>an invoice created in QuickBooks is imported</code> · <code>before the mapping exists: an invoice we created is linked, not imported twice</code>

Requisito	En vivo	Tests (nombre)
Actualizaciones	PATCH local; editar fecha en QuickBooks	only changed in QuickBooks: applied · only changed locally: kept, and pushed later against the latest version · a QuickBooks version conflict is retried with the latest version
Eliminación / void	DELETE local; Void y Delete en QuickBooks	deleting: deleted in QuickBooks, or voided if it has payments · deletions in QuickBooks · voided in QuickBooks: voided locally, not deleted · a failed GET is not taken as a deletion: the job is retried
Estado de pago	P1–P13; <i>Receive payment</i> en QuickBooks	paid status, both ways · partial payments recorded locally reach QuickBooks, and the balance comes back · payments made in QuickBooks are listed locally · deleting payments locally
Eventos duplicados	D1–D7; W2 dos veces	Idempotency-Key prevents duplicate invoices on retries · POST /invoices without an Idempotency-Key: 400, nothing created · without an Idempotency-Key: 400, nothing recorded · a duplicate delivery is recorded and queued once · a redelivered notification (same event key) is not queued again · our own push coming back is skipped (no sync loop)
Ediciones en conflicto	Worker detenido + editar en ambos lados	changed on both sides differently: conflict, nothing overwritten · same change on both sides: converges without a conflict · a local change does not overwrite a different change made in QuickBooks · 7. only the status changed locally, the content only in QuickBooks: no conflict, the status is pushed · 6. deleted during a conflict, resolved with "remote": still deleted in QuickBooks · 9. a conflict that converges ... is cleared, and what is left is pushed

4. Cómo se cubre cada supuesto del contexto

Supuesto	Cómo lo resuelvo	Test
Ambos sistemas exponen APIs	REST propia + Accounting API de QuickBooks (OAuth 2.0, tokens en Postgres)	suite completa
Ambos emiten notificaciones de cambio	Local: outbox + NOTIFY. QuickBooks: webhooks firmados (formato clásico y CloudEvents)	webhook endpoint
Eventos duplicados	event_key único, Idempotency-Key, requestid, SyncToken	ver fila "Eventos duplicados"
Eventos atrasados o desordenados	Se ignora un SyncToken igual o menor al guardado; los jobs de una misma factura corren en orden	an older version never overwrites a newer one · jobs of the same entity run one at a time, oldest first
Payload incompleto	El worker siempre vuelve a pedir la entidad completa	a change made in QuickBooks is applied locally (webhook path) (sandbox)

Supuesto	Cómo lo resuelvo	Test
Datos modificados fuera de la app (borrado a mano en la base)	Chequeo de consistencia periódico y a mano (<code>client.http</code> → Consistency check): reimporta lo que falta, confirma con GET antes de borrar	<code>an invoice deleted by hand in the database comes back, with its payments</code>
APIs que fallan o hacen timeout	Clasificación de errores, backoff + jitter, <code>UNKNOWN</code> + reconciliación, lease	<code>ambiguous create ... · retry scheduling · recovering jobs of workers that died</code>
Ediciones manuales en ambos lados	Webhooks + reconciliación CDC + detección de conflictos a tres vías	<code>three-way comparison with the last synced snapshot</code>

Apéndice A. Consultas para la consola de base de datos

```

-- A1. Facturas y su estado de sincronización
SELECT id, customer_name, amount, balance, status, quickbooks_id, sync_status, sync_error,
sync_conflict
FROM invoices ORDER BY id DESC LIMIT 10;

-- A2. Historial de sincronización de una factura (ambas direcciones)
SELECT direction, entity_type, operation, status, attempts, payload->'result' AS result,
last_error, created_at
FROM sync_jobs WHERE entity_key = 'invoice:123' ORDER BY created_at;

-- A3. Latencia: cuánto tardó el worker en tomar cada job (NOTIFY)
SELECT entity_key, round(extract(epoch FROM locked_at - created_at) * 1000) AS claim_ms,
duration_ms, payload->'result'
FROM sync_jobs WHERE direction = 'OUTBOUND' ORDER BY created_at DESC LIMIT 5;

-- A4. Avisos recibidos de QuickBooks (webhooks y reconciliación)
SELECT source, entity_type, external_entity_id, operation, processing_status, event_key,
received_at
FROM sync_events ORDER BY id DESC LIMIT 10;

-- A5. Pagos de una factura (registrados aquí o en QuickBooks)
SELECT id, amount, paid_on, quickbooks_id, sync_status, deleted_at FROM invoice_payments
WHERE invoice_id = 123;

-- A6. Estado de la cola
SELECT status, count(*) FROM sync_jobs GROUP BY status;

```

Los mismos datos por la API: `GET /sync/jobs?invoice_id=123`, `GET /sync/events`, `GET /sync/stats`, `GET /invoices/123/payments`.

Apéndice B. Preguntas probables y respuestas cortas

¿Por qué el Idempotency-Key es obligatorio y no opcional? Porque la protección tiene que estar en el primer intento: si el cliente no manda clave y el request hace timeout, al reintentar ya no hay forma de saber si la factura se creó. Con la clave obligatoria (400 sin ella, en facturas y pagos) todo reintento es seguro: la

misma clave devuelve lo creado la primera vez (200), con una columna única en la base. (Stripe, por ejemplo, la deja opcional; acá la hago obligatoria porque una factura duplicada es un error contable.)

¿Qué pasa si el usuario cambia la factura mientras el worker la está empujando? El worker guarda el resultado con la versión que empujó (`synced_version`). Si la factura cambió mientras tanto, queda `pending` y se encola otro job con el estado más nuevo. Una revisión de código encontró casos en los que el cambio nuevo se perdía igual (un "paid" que volvía a "sent", un borrado marcado como sincronizado); cada uno tiene ahora un test que lo reproduce en `test/sync.races.test.ts` .

¿Qué pasa si QuickBooks crea la factura pero se corta la conexión antes de la respuesta? El job marca `create_sent_at` justo antes de enviar; si no hay respuesta pasa a `UNKNOWN` y bloquea los demás jobs de esa factura. La reconciliación busca en QuickBooks la factura con nuestra referencia: una → se vincula; ninguna → se reenvía con el mismo `requestid` ; varias → `FAILED` para revisión manual. Nunca se reenvía a ciegas.

¿Y si dos workers toman el mismo job? No pueden: `FOR UPDATE SKIP LOCKED` . Y si un worker lento pierde su lease, el claim token impide que complete un job que ya tomó otro.

¿Cómo evitas loops (yo empujo, QuickBooks me avisa, yo vuelvo a empujar)? Después de empujar guardo el SyncToken que devuelve QuickBooks; el aviso de mi propio cambio trae ese mismo SyncToken y se descarta.

¿Por qué no aplicar el payload del webhook directamente? Puede estar incompleto, repetido o viejo. Siempre pido el estado actual a QuickBooks; así el orden de llegada deja de importar.

¿Y los impuestos? No los modelo: cada factura es una línea con un monto. Si QuickBooks le aplica sales tax, la factura queda marcada (`taxed_in_quickbooks`) y su contenido lo manda QuickBooks: acá no se edita (409), porque empujar una línea sin impuesto cambiaría el total. Estado, pagos y borrado siguen sincronizando. El siguiente paso sería modelar líneas.

¿Cómo manejas el dinero? `NUMERIC(14,2)` en Postgres y strings decimales en la API (`"1500.00"`). QuickBooks usa números JSON; con dos decimales la conversión es exacta para montos menores a 10^{13} .

¿Por qué polling si ya tienes LISTEN/NOTIFY? NOTIFY no es durable: si el worker está caído o reconectando, el aviso se pierde. La fuente de verdad es la tabla `sync_jobs` ; el worker la consulta cada 5 s (`waitForWork` + `claimNextJob`) y NOTIFY solo lo despierta antes. El costo del polling es una consulta indexada cada 5 s por worker, despreciable.

¿Qué pasa si se pierde un webhook? La reconciliación periódica usa el CDC de QuickBooks (cambios desde un cursor) y los encola por el mismo camino. Si la base local queda vacía, hace una importación completa; también si el worker estuvo apagado más de 30 días (lo que recuerda el CDC) o si el CDC devolvió su máximo de 1000 cambios por entidad.

¿Qué pasa si alguien borra una factura directamente en la base? QuickBooks no se entera, porque el borrado se salta el outbox. El chequeo de consistencia (cada 24 horas, o a mano con los requests **Consistency check** de `client.http`) compara todos los IDs con QuickBooks y la reimporta con sus pagos. En producción, además, nadie debería tener permiso de escritura directo sobre la base.

¿Cuándo usarías un broker? Con muchos consumidores del mismo evento, volumen muy alto o necesidad de replay. Aun así mantendría el outbox en Postgres como fuente, publicando al broker lo ya confirmado.